



Drools na prática - Parte 2: evoluindo o validador de CNH para uma API REST

No artigo anterior, comecei um estudo prático sobre o **Drools** criando um validador simples de CNH em Java.

A ideia era apresentar uma forma objetiva, como conceitos como **Facts**, **Working Memory**, **Agenda**, **KieSession**, **fireAllRules()** e **modify()** se relacionavam dentro de uma Rule Engine.

Naquela primeira versão, o fluxo era bastante simples:



O projeto cumpriu muito bem seu objetivo didático, mas, ao terminar aquela implementação, uma pergunta surgiu naturalmente:

e se, em vez de criar as CNHs diretamente no código, eu pudesse recebê-las através de uma requisição HTTP?

Foi a partir dessa pergunta que comecei a segunda etapa do projeto.

Desta vez, resolvi transformar meu exemplo em uma **API REST**, mantendo o Drools como responsável pelas decisões de negócio.

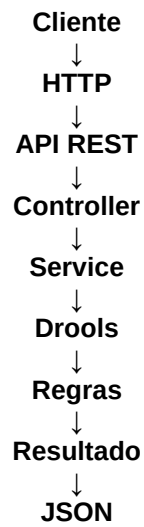
Do exemplo de console para uma API

No primeiro projeto, minha aplicação controlava praticamente todo o ciclo de execução, ela criava os objetos, inseria os fatos na sessão do Drools, executava as regras e mostrava o resultado.

Para estudar uma situação um pouco mais próxima de uma aplicação real, decidi mudar esse fluxo.



Agora eu queria algo assim:



O Drools deixou de ser apenas algo que era executado dentro de uma classe main e passou a fazer parte de uma **camada de serviço acessível externamente**.

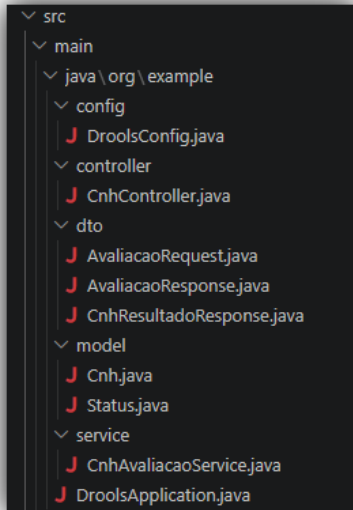
A partir daí qualquer aplicação capaz de realizar uma requisição HTTP poderia, conceitualmente, utilizar minhas regras.



A primeira mudança: Spring Boot entra no projeto

Para transformar o exemplo em uma API, você precisará estruturar o projeto como uma aplicação **Spring Boot**.

Na nova versão, minha organização assim:



Essa organização já traz uma diferença importante em relação à primeira versão.

Antes

CNH
+
Regra
+
Drools

Agora :

Entrada
+
Orquestração
+
Regra de negócio
+
Saída

Ou seja, comecei a separar melhor as responsabilidades da aplicação.



A regra continua sendo responsabilidade do Drools

Uma decisão que fiz questão de manter foi esta:

transformar o projeto em uma API não deveria significar mover as regras de volta para o Java.

Eu poderia facilmente criar algo assim dentro do Controller:

```
if (cnh.isExpirada()) {  
    ...  
}  
  
if (cnh.getPontos() > 20) {  
    ...  
}
```

Mas, se fizesse isso, perderia justamente o objetivo do projeto.

- **A API deveria cuidar do transporte.**
- **O Java deveria cuidar da orquestração.**
- **E o Drools deveria continuar cuidando das decisões.**

Minha arquitetura passou a representar algo próximo disto:



Essa separação começou a deixar o projeto muito mais interessante.

Criando uma camada de Controller

A primeira nova responsabilidade apareceu na classe:

CnhController

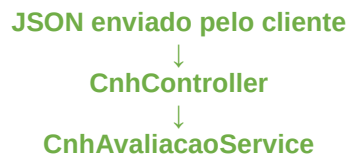
No primeiro projeto, eu não precisava de uma camada HTTP.

Agora preciso de um componente responsável por receber uma requisição e encaminhá-la para a aplicação.



Passei a enxergar o Controller como a **porta de entrada** do meu sistema.

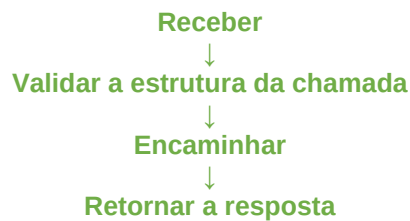
Conceitualmente:



Uma coisa que considero importante aqui é não transformar o Controller em uma classe cheia de regras.

➤ **o Controller deve permanecer simples.**

Sua responsabilidade é algo próximo de:



Ele não deveria decidir se uma CNH está válida ou inválida.

Essa decisão continua pertencendo às regras.

Por que criei DTOs?

Na primeira versão, eu trabalhava diretamente com meus objetos Cnh.

Ao transformar o projeto em uma API, comecei a separar também os objetos utilizados internamente dos objetos utilizados na comunicação HTTP.

Foi aí que entraram classes como:

- **AvaliacaoRequest**
- **AvaliacaoResponse**
- **CnhResultadoResponse**

Essa mudança mostra melhor a funcionalidade do DTO de maneira simples.

DTO — Data Transfer Object.



Imagine que o modelo interno possuía:

- **Cnh**
- **numero**
- **nome**
- **pontos**
- **dataExpiracao**
- **status**
- **causa**

Nem sempre quero que o cliente da API tenha controle sobre todos esses campos.

Por exemplo, status e causa fazem muito mais sentido como **resultado da avaliação** do que como uma decisão fornecida pelo cliente.

Eu posso receber algo conceitualmente semelhante a:

```
{
  "numero": "123456789",
  "nome": "João",
  "pontos": 10,
  "dataExpiracao": "2027-10-20"
}
```

E deixar minha aplicação determinar:

```
{
  "numero": "123456789",
  "nome": "João",
  "status": "VALIDA",
  "causa": "Nenhuma regra impeditiva"
}
```

Essa separação começa a criar um contrato mais claro para a API.

A camada de Service virou a ponte com o Drools

Uma das mudanças que mais legal nessa evolução foi a criação da classe:

CnhAvaliacaoService

Na primeira versão, o fluxo relacionado ao Drools estava muito próximo da execução principal da aplicação.



Agora você consegue concentrar a avaliação dentro de um serviço.



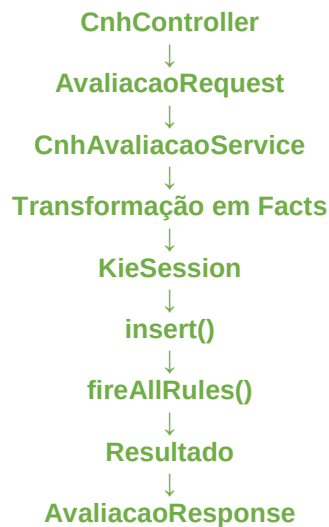
O Controller não precisa saber exatamente como o Drools funciona.

Ele simplesmente pede:

"Avalie estas CNHs"

O Service fica responsável por coordenar essa operação.

Conceitualmente, o fluxo passou a ser:



Isso melhorou bastante a separação de responsabilidades do projeto.

O Drools ganhou uma configuração própria

Outra evolução foi retirar a responsabilidade de configurar o Drools do fluxo principal da aplicação.

Para isso, passei a ter uma classe específica:

DroolsConfig

Essa classe existe para concentrar a configuração necessária para que os componentes do Drools possam ser disponibilizados para o restante da aplicação.



Antes eu tinha algo muito próximo disto:



Agora passei a pensar em:



Com isso, minha lógica de avaliação não precisa ficar recriando toda a infraestrutura de configuração.

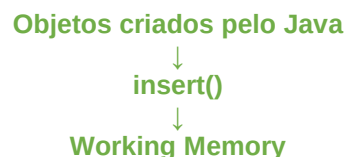
A aplicação começa a tratar o Drools como uma dependência da camada de serviço.

Mas o que aconteceu com a Working Memory?

Ela continua existindo, essa foi uma percepção interessante durante a evolução do projeto. Adicionar uma API REST **não altera os conceitos fundamentais do Drools**.

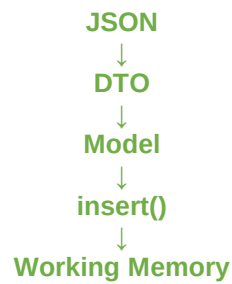
Eu apenas mudei a forma pela qual os fatos chegam até ele.

Antes:





Agora:



Da perspectiva do Drools, no final das contas, ele ainda recebe Facts.

A origem desses Facts mudou e isso é bastante interessante arquiteturalmente.

O motor de regras não precisa saber se aquela CNH veio de:

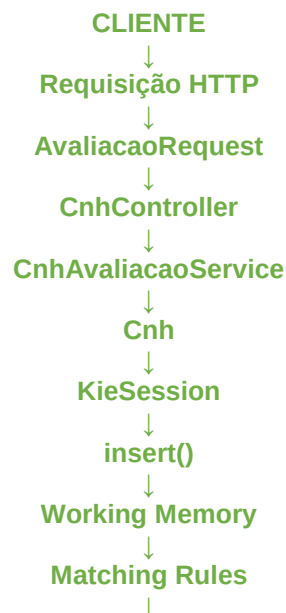
- *um console*
- *um banco de dados*
- *uma API REST*
- *uma fila*
- *um arquivo*
- *outro sistema*

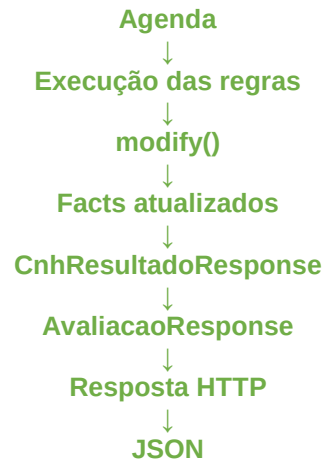
Ele recebe um objeto e avalia suas condições.

Essa independência é uma das características que comecei a valorizar mais nessa segunda versão.

O fluxo completo da nova aplicação

Se no primeiro artigo meu fluxo terminava praticamente no console, agora consigo visualizar a aplicação desta forma:





Para mim, essa é a principal evolução entre os dois projetos.

- O núcleo do Drools permanece praticamente com a mesma responsabilidade.
- O que cresceu foi a arquitetura ao redor dele.

As regras continuam pequenas

Outro ponto que fiz questão de preservar foi a simplicidade do arquivo `rules.drl`.

Continuo trabalhando com regras como `Cnh(isExpirada())` para identificar uma CNH expirada e `Cnh(pontos > 20)` para identificar excesso de pontos.

Além da regra que observa o estado:

```
status == Status.NAO_PROCESSADA
```

O ponto importante é que transformar o projeto em API não exige que as regras conheçam:

- HTTP
- JSON
- Controller
- Spring
- DTO
- Response

E, para mim, isso é uma boa indicação de separação de responsabilidades.

- A regra continua falando sobre **CNH**.
- Não sobre infraestrutura.

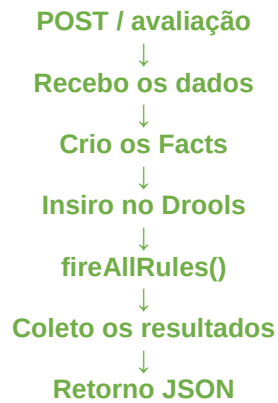


Uma requisição agora dispara uma avaliação completa

Na primeira versão, eu executava a aplicação e observava o resultado no console.

Nesta versão, passei a enxergar cada chamada da API como uma nova avaliação.

Conceitualmente:



Isso também torna o exemplo muito mais fácil de integrar a outros sistemas.

- Um front-end poderia consumir a API.
- Outro back-end poderia chamá-la.
- Uma aplicação mobile também.

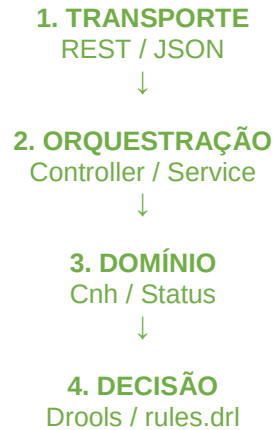
O cliente não precisa saber que existe Drools por trás da API, para ele, o contrato é HTTP.

Por dentro, a decisão continua orientada a regras.



Uma separação que comecei a considerar fundamental

Depois dessa segunda versão, passei a organizar mentalmente a aplicação em quatro partes.



Essa separação tornou o projeto muito mais claro, se amanhã eu alterar uma regra de pontos, teoricamente não preciso alterar o Controller.

- Se eu alterar a estrutura da API, não deveria precisar mudar a lógica da regra.
- Se eu trocar a origem dos dados, o Drools também não deveria precisar saber.

Esse desacoplamento é justamente uma das coisas que eu queria observar quando comecei este estudo.

O que mudou na visão sobre o `fireAllRules()`

No primeiro projeto tínhamos o `fireAllRules();`, como o momento em que o Drools simplesmente começava a processar meus objetos.

Agora esse método também é uma espécie de **fronteira entre a aplicação e o motor de decisões**.

Antes dele:

- HTTP
- DTO
- Service
- Model



Depois dele:

- **Working Memory**
- **Agenda**
- **Rules**
- **Activations**
- **modify()**

Finalmente, a aplicação pega o resultado e o transforma novamente em algo adequado para quem fez a requisição.

E a KieSession?

Aqui também aparece um ponto arquitetural importante, no projeto inicial, trabalhar com uma sessão era relativamente simples porque eu tinha um único fluxo executando e terminando.

Quando transformo isso em uma API, começo a precisar pensar também em questões como:

- *ciclo de vida da sessão*
- *isolamento entre requisições*
- *concorrência*
- *estado*
- *uso de memória*
- *dispose()*

Essa é uma diferença importante entre um **projeto didático** e uma aplicação que realmente começaria a receber múltiplas chamadas.

Eu não quero que os fatos de uma requisição acabem interferindo nos fatos de outra.

Por isso, uma evolução natural desse estudo é você entender muito bem qual estratégia de sessão é mais apropriada para cada cenário.

O projeto deixa de ser somente:

“Minha regra funciona?”

e começa a incluir a perguntar:

“Minha Rule Engine está sendo utilizada corretamente dentro de uma aplicação web concorrente?”

Para mim, essa é uma das próximas fronteiras interessantes do estudo.



A API também me faz pensar mais sobre contratos

Quando eu executava tudo localmente, podia simplesmente imprimir:

RENAN - INVALIDA - CNH expirada

Dentro de uma API, preciso pensar melhor no formato da resposta.

Por isso, classes como `AvaliacaoResponse` e `CnhResultadoResponse` passam a ter importância.

A regra produz uma decisão, mas quem consome a API precisa receber essa decisão de maneira previsível.

Por exemplo:

```
{
  "resultados": [
    {
      "nome": "RENAN",
      "status": "INVALIDA",
      "causa": "CNH expirada"
    },
    {
      "nome": "REBECA",
      "status": "VALIDA",
      "causa": "Nenhuma regra impeditiva"
    }
  ]
}
```

A partir daqui começo a entrar em outro assunto importante:

uma Rule Engine resolve a decisão de negócio, mas não elimina a necessidade de projetar uma boa API.

São problemas diferentes.

O Drools não virou aplicação inteira

Essa talvez seja uma das conclusões mais importantes dessa evolução, é fácil, quando estamos estudando determinada tecnologia, começar a enxergá-la como o centro de tudo.

Mas, ao transformar o projeto em API, ficou ainda mais claro que o Drools tem uma responsabilidade específica.



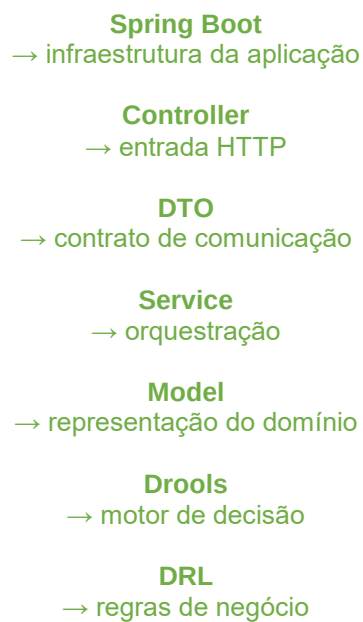
Ele não precisa:

- *receber HTTP*
- *serializar JSON*
- *controlar rotas*
- *tratar autenticação*
- *persistir dados*
- *montar responses*

Ele precisa fazer muito bem aquilo para que foi escolhido:

- **Avaliar e tomar decisões baseadas em regras de negócio.**

Minha arquitetura passou a ficar mentalmente assim:





O que eu ganhei transformando o projeto em API?

A principal vantagem não foi simplesmente conseguir utilizar Postman, curl ou outra aplicação para realizar uma chamada, o ganho maior saber que a **Rule Engine** pode ser colocada atrás de uma interface bem definida.

Agora posso imaginar:



Todos esses consumidores podem utilizar as mesmas regras, isso também abre espaço para um cenário ainda mais interessante.

Imagine que amanhã a regra, **pontos > 20** mude, a alteração ocorre no conjunto de regras.

Os consumidores continuam chamando o mesmo contrato da API.

Essa separação entre **consumo e decisão** começa a demonstrar melhor onde uma Rule Engine pode fazer sentido em uma arquitetura.

E tem como melhorar?

Obviamente que sim, aqui está uma dica:

- ➔ Transformar o exemplo em API não resolve todos os problemas, na verdade, cria novos assuntos para estudar.

Você deve pensar em questões como:

- *validação dos dados recebidos*
- *tratamento de exceções*
- *respostas HTTP adequadas*
- *concorrência*
- *ciclo de vida da KieSession*
- *testes da API*
- *testes das regras*
- *versionamento das regras*
- *observabilidade*
- *logging*
- *segurança*

Também continua existindo a questão que identifiquei no primeiro artigo:

- ➔ as regras possuem interação entre si, lembra?



Minha regra de CNH válida observa:

```
status == Status.NAO_PROCESSADA
```

Enquanto outras regras alteram exatamente esse estado.

Portanto, ainda uma boa prática é cuidar da estratégia de execução e da forma como as regras se relacionam.

- ➔ A API não elimina esse problema.
- ➔ Ela apenas coloca o motor de regras dentro de um contexto arquitetural maior.

O próximo passo que eu faria: testar cada camada separadamente

Depois dessa evolução, um dos próximos pontos que considero fundamental é criar uma estratégia de testes dividida por responsabilidade.

Eu não deixo tudo apenas fazendo uma requisição HTTP e verificando o resultado.

- ➔ Procure testar a regra diretamente.

Por exemplo:

DADO
uma CNH expirada

QUANDO
eu executar as regras

ENTÃO
ela deve ficar INVALIDA

Depois:

DADO
uma CNH com 25 pontos

QUANDO
eu executar as regras

ENTÃO
ela deve ficar INVALIDA



E também testar a API:

DADO
um JSON válido

QUANDO
eu chamar a avaliação

ENTÃO
devo receber HTTP 200
e um resultado estruturado

Assim comece a separar:

Teste de regra
≠
Teste do Service
≠
Teste do Controller
≠
Teste de integração

Essa distinção passa a ser muito importante conforme o projeto cresce.

Outra evolução interessante: regras fora do deploy da aplicação

Ao chegar nesse ponto, surgiu outra pergunta que considero ainda mais interessante:

➔ **e se uma alteração de regra não precisasse obrigatoriamente de um novo deploy da API?**

Hoje posso manter minhas regras dentro da própria aplicação, mas, em sistemas maiores, posso começar a estudar estratégias de:

- versionamento das regras
- repositórios de regras
- KJAR
- KIE Server
- Business Central
- atualização controlada de regras

Nesse cenário, comece a avançar de:

"uma aplicação Java que usa Drools"

para algo mais próximo de:

"uma arquitetura de gerenciamento de decisões"



Comparando as duas versões

Quando olho para o primeiro e para o segundo projeto, percebo claramente a evolução.

Primeira versão



O objetivo era entender a Rule Engine.

Segunda versão



Agora o objetivo passou a ser entender **como incorporar essa Rule Engine dentro de uma aplicação de serviço**.

E considero que essa mudança foi muito mais importante do que simplesmente “adicionar Spring Boot”.

Ela mudou o papel do projeto.

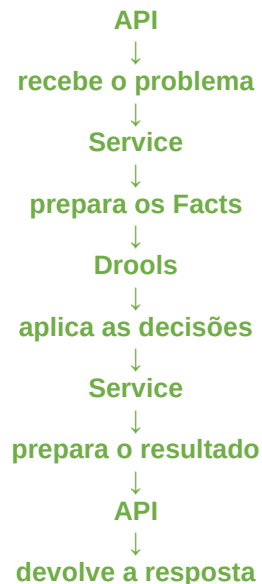
No primeiro artigo, procurei mostrar a você que o Drools não deve ser enxergado simplesmente como um conjunto de if/else escritos em outro arquivo.



Nesta segunda etapa, meu principal aprendizado foi outro:

→ o motor de regras funciona melhor quando possui uma responsabilidade bem delimitada dentro da arquitetura.

Hoje eu visualizo:



O Drools não precisa conhecer quem fez a chamada.

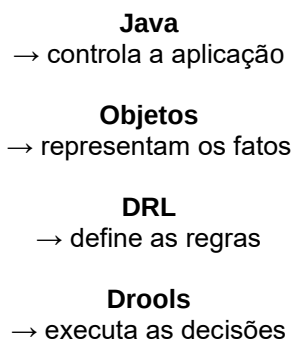
- O cliente não precisa conhecer o Drools.
- E minha regra não precisa conhecer HTTP.

Isso representa uma evolução importante na arquitetura.

No primeiro projeto, tiramos o Drools da teoria, vimos o que acontecia quando um Fact entrava na **Working Memory**, como uma regra era ativada, qual era o papel da **Agenda** e por que `modify()` fazia diferença.

Depois de entender esse fluxo, transformamos o exemplo em uma API foi um próximo passo natural, exatamente a evolução que eu havia deixado como possibilidade ao final do artigo anterior.

Antes:





Agora eu acrescentaria:

REST

→ disponibiliza o serviço

DTO

→ define o contrato

Controller

→ recebe as requisições

Service

→ orquestra a avaliação

Drools

→ executa as decisões

Response

→ devolve o resultado

A regra de negócio continua sendo praticamente a mesma, o que mudou foi tudo que construí ao redor dela.

E talvez esse tenha sido o principal aprendizado que trouxe nesta segunda etapa:

→ uma Rule Engine começa a ficar ainda mais interessante quando deixa de ser apenas um experimento isolado e passa a ocupar um lugar bem definido dentro da arquitetura de uma aplicação.

EducaCiência FastCode para a comunidade



Bora de Print da funcionalidade Drools – API

```
src > main > java > org > example > DroolsApplication > @mainString()
1 import org.springframework.boot.autoconfigure.SpringBootApplication;
2
3 @SpringBootApplication
4 public class DroolsApplication {
5
6     private static final Logger log = LoggerFactory.getLogger(DroolsApplication.class);
7
8     public static void main(String[] args) {
9         SpringApplication.run(DroolsApplication.class, args);
10    }
11
12    @PostConstruct
13    public void init() {
14        log.info("Versão Drools: 7.7.0.Final");
15        log.info("Regras: package org.example -> rules.drl");
16        log.info("Endpoint: http://localhost:8082/api/cnh/avaliar");
17    }
18 }
```

```
POST http://localhost:8082/api/cnh/avaliar
Content-Type: application/json

[{"cnh": {"numero": "CNH-100", "nome": "REBECA", "pontoss": 5, "dataExpiracao": "2028-06-30"}, {"cnh": {"numero": "CNH-200", "nome": "ROBERTA", "pontoss": 10, "dataExpiracao": "2029-04-30"}, {"cnh": {"numero": "CNH-250", "nome": "FABIO", "pontoss": 25, "dataExpiracao": "2029-04-30"}]}

{"regrasExecutadas": 3, "resultados": [{"numero": "CNH-100", "nome": "REBECA", "pontoss": 5, "dataExpiracao": "2028-06-30", "status": "VALIDA", "causa": "Cnh sem status, considerada valida por enquanto"}, {"numero": "CNH-200", "nome": "ROBERTA", "pontoss": 10, "dataExpiracao": "2029-04-30", "status": "VALIDA", "causa": "Cnh sem status, considerada valida por enquanto"}, {"numero": "CNH-250", "nome": "FABIO", "pontoss": 25, "dataExpiracao": "2029-04-30", "status": "INVALIDA", "causa": "Cnh tem mais pontos que o permitido de 20"}]}
```

```
POST http://localhost:8082/api/cnh/avaliar
Content-Type: application/json

[{"cnh": {"numero": "CNH-250", "nome": "FABIO", "pontoss": 25, "dataExpiracao": "2029-04-30"}]}

{"regrasExecutadas": 1, "resultados": [{"numero": "CNH-250", "nome": "FABIO", "pontoss": 25, "dataExpiracao": "2029-04-30", "status": "INVALIDA", "causa": "Cnh tem mais pontos que o permitido de 20"}]}
```